



Engineering a cognition-based specification method

Robert Deckers^{1,2} · Patricia Lago¹

Received: 28 June 2024 / Revised: 19 March 2026 / Accepted: 6 May 2026
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2026

Abstract

Context Software development is inherently a human cognitive task that involves the capture and integration of diverse knowledge and decisions from multiple stakeholders. Existing specification methods and languages mostly rely on computer-based or mathematical primitives, leading to a disconnect between how people naturally think and communicate, and how systems are specified. Therefore, we are investigating a method, called MuDForM (Multi-Domain Formalization Method), to formalize and integrate the knowledge of multiple domains into domain models and into specifications in terms of those domain models. We created a first coherent definition of the method, which emerged from several case study evaluations published in previous works.

Goal Establish a method definition that is explicitly based on concepts from human cognition.

Method We studied literature in (language) philosophy, linguistics, and cognitive science, to identify concepts that can serve as the cognitive underpinning of the method's metamodel. We made a model of those concepts and connected them to MuDForM's metamodel via an explicit method definition structure and analyzed the result.

Result The paper defines the conceptual and structural groundwork for illustrating how a specification method can be constructed based on insights into human cognition and communication. It provides a coherent model of cognitive specification aspects, which grounds the modeling concepts in MuDForM's metamodel and makes it cognition-based. We also identified additional cognitive aspects that call for future work and the possible extension of the metamodel. The paper clarifies MuDForM's objective to support the transformation of natural language into unambiguous, cognition-aligned models.

Keywords Method engineering · Specification method · Domain modeling · Cognition · Software engineering

1 Introduction

Many specification methods and languages exist for system development and for software development in particular *e.g.*, [1–5] and the overview of Frappier [6]. We envision software development as a process in which the people involved capture decisions in a language close to their area of knowledge, *i.e.*, *domain*, and in which those decisions are integrated, so that they finally result in a machine-readable specification. That is why our research focuses on an integral method,

called MuDForM (Multi-Domain Formalization Method), for creating domain models, for using domain models as a language to create other (domain-based) specifications, and for integrating multiple domain models and domain-based specifications. We created a first coherent definition of the method, emerging from previous evaluation in several published case studies [7–12].

This paper specifically addresses how the definition of MuDForM's metamodel is grounded in concepts from human cognition. To this end, we have studied literature in the fields of (language) philosophy, linguistics, and cognitive science and analyzed the findings, which are recorded in an elaborate report [13]. The report identifies so-called cognitive specification aspects, which are used as the foundation of MuDForM's metamodel.

The rest of this paper is structured as follows: Section 1.1 describes the problem statement and the contribution. Section 2 explains the vision behind our research, and the works that form the context of this paper. Section 3 explains the fol-

Communicated by Jeff Gray.

✉ Patricia Lago
p.lago@vu.nl
Robert Deckers
robert.deckers@AtomFreeIT.com

¹ Faculty of Science, Vrije Universiteit Amsterdam, Amsterdam, The Netherlands

² Atom Free IT, Heeswijk-Dinther, The Netherlands

lowed research methodology. Section 4 gives an overview of the method definition structure. Section 5 explains the cognitive specification aspects on which the method's metamodel is based. Section 6 explains MuDForM's definition and how it relates to these aspects. Section 7 discusses the related work, and Sect. 8 discusses the results. Section 9 presents conclusions and topics for future work.

1.1 Problem and contribution

We conducted a Systematic Literature Review [14] (SLR), which concluded that most specification methods and languages used in software development focus on describing system functionality or software design [1, 4, 15, 16]. Even specification languages that can be used for any aspect of a system or its environment, *e.g.*, domain-specific languages (DSLs), are sometimes positioned as a programming language [17].

To make specifications that are not tied to functionality or software design, we investigate the use of concepts from human cognition and human communication as the basis of a language and method, starting from, but not limited to, natural language. The problem addressed in this paper is the absence of a widely accepted specification method or language that is explicitly based on cognitive concepts, which is suggested by at least two publications.

Siau [18] states that despite the pivotal role of modeling methods, most method designs are based mainly on the common sense and intuition of method designers. He proposes to use cognitive psychology as a reference discipline for information modeling and method engineering. He discusses the different types of memory and knowledge and gives an example of how they can impact and restrict a method design, but does not concretely state how they can be used to define method ingredients.

Wand [19] distinguishes ontological elements and meta-model elements and demonstrates and concludes that they can be related, but should not necessarily be the same. Wand does not state what the proper ontological elements are, nor does he define an approach to find them.

The contributions of this paper are organized in three sections: Section 4 presents a structure for defining a specification method that is driven by method objectives and based on cognitive aspects. This structure is applied for the definition and evolution of MuDForM and can be used by developers of other methods. The structure frames the contributions described in Sects. 5 and 6. Section 5 presents a model of cognitive aspects that can be addressed in specifications. The model can be used as a foundation of specification-related artifacts, such as specification methods, and forms a baseline for future extensions with more cognitive aspects. Section 6 explains how the cognitive aspects of Sect. 5 ground the definition of MuDForM's metamodel, which can be used

for further development of MuDForM or other cognition-based methods. Overall, this paper shows an approach by which (software development) method engineering can be connected to cognitive science.

2 Background: MuDForM vision and development

The overarching vision behind MuDForM is that system development, and software development in particular, can be seen as the integration of knowledge and decisions from multiple people responsible for different aspects. With this in mind, we started to develop MuDForM; a method based on (human) cognitive concepts instead of computer-based or mathematical concepts. Appendix B of the background report [13] and the MuDForM positioning report [20] describe the vision in more detail.

The vision has been elaborated into eight objectives [11, 14] that guide the development of MuDForM. The objectives formed the starting point of our SLR [14] of the existing literature on domain-oriented specification methods. The SLR revealed, among other things, that thorough methodical support for processing natural language to create models is hardly addressed in the existing literature.

This paper particularly focuses on one of the eight objectives: A specification method should support **the transformation of knowledge and decisions expressed in natural language into specifications in an unambiguous language that is based on cognitive concepts**.

Almost everyone, including domain experts, uses natural language to convey their knowledge and decisions. It is used in many documents in any software development process. Sigman [21] clarifies that the awareness of natural language is innate. Even newborn babies recognize the language spoken by their mother. Levithin [22] discusses the lexical hypothesis, *i.e.*, the most important things humans need to talk about eventually become encoded in language. This is a motivation for using natural language as entry point for a development process.

Natural language is a starting point, but probably does not cover all relevant cognitive concepts, because it evolved for spoken communication and not specifically for thinking and specifying systems. Pinker [23, 24] discusses *Mentalese*: the internal language of the mind, which is free from syntax. The mind can think without having words. People who did not learn language can still think and solve problems; just like some animals. Pinker [25] also discusses the misconception that the native language of a person determines what they can think. This misconception is called *genuine linguistic determinism*, aka the Whorfian¹ hypothesis. According to

¹ https://en.wikipedia.org/wiki/Benjamin_Lee_Whorf.

Pinker, language clearly influences thoughts. Otherwise, it would be useless. Nevertheless, Evermann [26] claims that the psychology of language is an appropriate area of research to discover human cognitive structures.

The mentioned SLR also concluded that the KISS method [3] is the closest to achieving the method objectives. Therefore, it is used as the starting point for the definition of MuDForM. MuDForM uses the following characteristics of the KISS method: (1) separation between domain and feature, explained in Sect. 6.1, (2) activities as first-class citizens, including the relationship between activities and classes, exemplified in Sect. 6.3, and (3) use of natural language processing in making specifications, clarified in Sect. 6.4.

According to Kronl f [27], a method definition should contain the following ingredients: an underlying metamodel of modeling concepts, a notation (encompassing multiple viewpoints), method steps, and guidance for taking steps and making decisions. We defined initial versions of the method ingredients [28], because the existing literature on the KISS method [3, 29, 30] does not provide them. The metamodel and method steps are the result of more than 30 years of experience in creating, using, and maintaining models of various domains and systems. We have recorded and consolidated our experiences since we started the MuDForM research in 2015. We actively manage the metamodel, steps, and guidelines in an online-available model [28].

Together with industry experts, we applied MuDForM in several case studies [8–11]. We recorded examples of major analysis and modeling decisions, which led to new and revised guidelines. We also refined the method steps and their descriptions and identified new viewpoints. The research and development of MuDForM and related publications resulted in a PhD thesis [12], which reflects the scientific aspect of the work.

This paper highlights how the modeling concepts in the metamodel are based on concepts from human cognition. The next section explains the research methodology that we followed.

3 Research methodology

This section describes the research methodology that led to the results in this paper, which addresses the following research question:

How to define a specification method that is explicitly cognition-based?

The methodology for this paper fits within the MuDForM development approach described in Sect. 2 and can be characterized as qualitative data analysis combined with design science.

First, we created a method definition structure (see Sect. 4).

Second, we identified relevant cognitive concepts in the literature through qualitative data analysis. To find relevant literature on philosophy, linguistics, and cognitive science, we started reading the primary studies from our SLR [14] that addressed the use of natural language, and books on language philosophy and linguistics by Noam Chomsky [31], Steven Pinker [23], John McWhorter [32], and John Searle [33]. A snowballing approach (as described by Jalali and Wohlin [34]) was used to expand the reading list, which ultimately included 150 books and lectures. During reading, we bookmarked passages that we considered usable for the definition of a method ingredient. Twenty-nine books and lectures yielded approximately 80 usable *bookmarks*. Each bookmark led to a *finding*, *i.e.*, an observation, consideration, or conclusion, and an *implication* for MuDForM. Then, the resulting triplets (bookmark, finding, implication) were organized into 69 reflection snippets. The snippets were grouped into reflection themes and recorded in the background report [13].

Third, we created a model of the identified cognitive concepts that can be expressed in specifications (see Sect. 5) and consequently served as underpinning for modeling concepts. We call them cognitive specification aspects; *cognitive aspects* for short. Section 6 explains how we used the method definition structure to connect the cognitive aspects to the modeling concepts in MuDForM's metamodel.

The underpinning of method ingredients with cognition-based concepts did not directly lead to new modeling concepts. However, we identified several issues and opportunities for new modeling concepts, which we discuss in Sect. 8.

The goal of this approach is that the metamodel's closeness to cognition is not merely a claim, but is systematically built in via the explicit relation between cognitive aspects and modeling concepts. For example, the cognitive aspect Category grounds the definition of the modeling concept Classifier.

4 Method definition structure

This section explains a structure for method definitions. The origin of the structure is defined by Kronl f [27]. Accordingly, a method definition contains the following ingredients (see Fig. 1):

- An underlying model, *e.g.*, metamodel, core model, or abstract syntax, that defines the modeling concepts, *i.e.*, the elements of the method's modeling language, and the semantics of a model.
- Explicit method steps that guide the modeler from eliciting knowledge from documents and experts to making a consistent and complete model. Each step involves several modeling concepts and one or more viewpoints.

- Guidance for taking steps and making decisions during the steps.
- An explicit notation, possibly used in multiple viewpoints, defined in terms of the modeling concepts.

For the engineering of MuDForM, we added three additional elements (the gray classes in Fig. 1). First, each method ingredient supports a number of *method objectives* that the method definition aims to realize. Second, we use the notion of *cognitive aspect* to give modeling concepts a cognitive foundation. Third, we have applied several *construction patterns* and the notion of specialization from UML [35] to ensure uniformity in the definitions of modeling concepts. Consequently, modeling concepts can be based on cognitive aspects, are formed by applying construction patterns, and can have parent classes. This paper focuses on the cognitive aspects and how they are the basis for MuDForM's modeling concepts and construction patterns, and clarifies the use of natural language in the specification process. To help the reader navigate through the rest of the paper, some classes in the diagram point to the sections where their instantiation is explained.

5 Cognitive aspects

This section explains the concepts that form the cognitive underpinning of the MuDForM modeling concepts, *i.e.*, the cognitive aspects derived from the studied literature. They represent concepts that the human mind uses to think and communicate and that can be addressed in specifications. They are captured in one coherent model, which is visualized by the diagrams of Fig. 2 and Fig. 3.

Sections 5.1–5.10 explain the cognitive aspects of Fig. 2, and Sect. 5.11 explains Fig. 3. As it is not a very familiar UML notation, we mention that the role names on the association ends are used in the subsets qualifications. For example, the association *identifies*² (see the left side of Fig. 2) can be seen as a specialization of *refers to*, and the association *changes* can be seen as a specialization of *involves*.

5.1 Concept

The root of the diagram is the class *Concept*, which stands for something that the human mind can give meaning to. Examples are “my car”, “the Eiffel Tower”, and “the fear of spiders”. It is derived from the definition of concept from Gottlob Frege³ as explained by John Searle [36]: A concept is a function with a truth value, and at least one argument. The statement “the king of France is bald” is not a concept

because it is neither true nor false. It simply does not mean anything. To be clear: baldness exists, but France does not have a king. That is why the statement is not a concept.’

Concept is the most fundamental class in the meta-model; it is the root of most other cognitive aspects. *Concept* does not stand alone, because a specification requires at least a *Symbol* to refer to a *Concept*, similar to the language triangle of Chomsky [31] and the notion of *Mentalese* discussed by Pinker [23, 24]. The subclasses of *Concept* are non-disjoint, *e.g.*, something that is a *Property* can also be an *Entity*.

5.2 Category

A *Category* is a *Concept* that categorizes other *Concepts* as elements of the *Category*. Examples are “cars”, “things that fit in my car”, and “animals that I fear”. Categories are ubiquitous in human thought, which is, among others, observed by Pinker [37] and Hofstadter and Sander [38]. Categories also enable inference of properties: if an object belongs to a category, one can infer its properties based on the category’s definition, as observed by Pinker [24]. Pinker clarifies: ‘if Mopsy is a rabbit, he should eat carrots, go hippity-hop, and breed like, well, a rabbit’.

Typically, nouns and verbs are *Symbols* in natural language that refer to *Categories*. For example, “chair” refers to a set of things that we consider a chair. Categories are not limited to things that can be referred to by a single word. For example, the category “things in my refrigerator” does not have a designated English word for it. Categories are fundamental in all specification (and programming) languages. Operations, functions, attributes, classes, and data types are all categories.

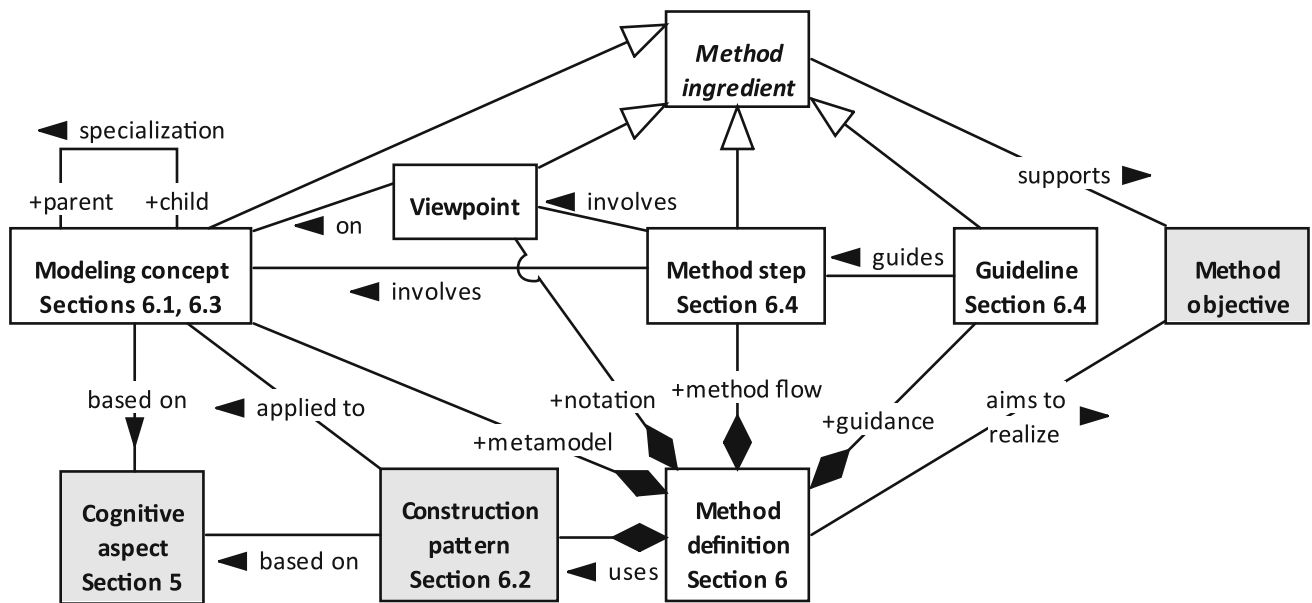
5.3 Entity

An *Entity* is a *Concept* identified by at least one *Symbol*. *Entities* are not dependent on any *Category*. So, *Entities* exist without being explicitly categorized by a *Category*. The aspect *Entity* brings the possibility of distinguishing things as separate *Concepts* that we can refer to through their identifying *Symbol* (name, number, picture of it, pointing to it, etc.). For example, “my car” is identified by “its chassis number”. Or, in the context of my possessions and the knowledge that I own only one car, I can use the *Symbol* “my car” to identify my car.

Searle [36] and Pinker [25] point to the difference between being countable and having identity. Two objects are identical if and only if they have the same properties. MuDForM follows this rule, called Leibniz’s law, which is reflected through *Entity*: two real-world objects without distinguishing properties correspond to one model instance. For

² From here on, cognitive aspects are written in typescript font.

³ https://en.wikipedia.org/wiki/Gottlob_Frege.



example, two plastic cups that look exactly the same may correspond to one model instance that has the properties of that type of plastic cup. To distinguish two cups, they must have distinctive properties, which, for example, could be their locations.

5.4 Event

An Event is something that happens (in time). Events can be indivisible in time, *i.e.*, they are atomic, or they can have a distinct start and stop moment. An Event may involve zero or more Concepts. Examples are “I am driving my car”, “I see my car on the driveway”, and “It rains”.

Several works discuss that people perceive time through Events, *e.g.*, Pinker [25], Corballis [39], and Sigman [21]. Pinker refers in another work [40] to the 1748 book on human understanding of philosopher David Hume⁴: ‘There appear to be only three principles of connections between ideas: similarity, contiguity in time or place, and cause or effect’.

An Event may have some order (in time) in relation to other Events, like sequentially, or simultaneously, or one Event happens during another Event, which reflects contiguity in time. Examples are “I see my neighbor while driving” and “I put my seatbelt on, before I start my car”.

An Event may cause other Events. Perceiving causality is very common for people (and other animals), which is, for example, discussed by Shermer [41] when he explains patternicity. Seeing patterns has evolved as a mechanism for survival and reproduction, especially to relate cause and effect.

An Event can be performed by an Agent, which also reflects causation. Not all Events are explicitly performed by an Agent. For example, when the Event “I knock the glass off the table” causes the Event “the glass falls on the floor”, who would be the Agent of the second Event? It is strange to state that the first Event is the Agent of the second Event. However, there is clearly a relation between causes and performs. There is also a relation between causes and order, because an Event cannot be caused by an Event that happens later.

5.5 Action

An Action is an Event that changes one or more Objects. Changes is a specialization of involves. An Action may also involve Concepts that are not an Object. Examples are “I am starting my car with the key”, in which the car changes state, but the key itself does not. Or, “my car accelerates to 100km/hour”, in which my car changes state, but “100km/hour” does not.

Sloman and Fernbach [42] mention different reasons why thought evolved, which all have one thing in common: thought evolved for action. Thought is helpful for predicting the effect of actions and reasoning about actions that we performed. Evolutionarily seen, action came before thought. Pinker [37] observes that the verb “to do”, among others, is present in most languages, which is covered by the cognitive aspect Action.

5.6 Object

An Object is an Entity that is changed by Actions. An Object exists over a period; therefore, we can speak of its state. Examples are “I started my car, and now the engine is running.” Or, “I step into my car, which reduces the number of free seats by one.” One can say that an Object is alive when it can still be changed by Actions, and dead when it cannot be changed anymore. A dead Object can still be involved in Events, but such an Event is just not changing the state of the Object.

5.7 Agent

An Agent is a Concept that performs Events and perceives Concepts. Pinker [24] observes that one of the fundamental metaphors in language is ‘force, agency, and causation’. An Agent performing Events reflects force. Force is implicitly also present in the order relation, *i.e.*, if a specification states that one Event must be followed by another, then this can be seen as an expression of force. In addition, the combination of an Agent perceiving an Event, followed by that Agent performing an Event can be seen as force.

The associations performs and perceives are both a specialization of involves, which is expressed in the diagram via the subset annotations. Examples are “I start (perform) my car” and “I see (perceive) that my car is blue”.

The definitions of Agent, Action, and Object are significant to the human mind, because people, *i.e.*, Agents, perform Actions to establish some change in Objects.

5.8 Property

A Property is a Concept that is a characteristic of another Concept. In other words, a Property is a Concept that belongs to, is part of, or is used to define another Concept. Examples are “my car is blue”, “cars are vehicles”, and “cars have wheels”. Typically, properties are used to express composition or hierarchy, or to determine time, location, or possession.

⁴ https://en.wikipedia.org/wiki/David_Hume.

Hofstadter and Sander [38] observe that concepts are hierarchical. We are building concepts from other concepts throughout our entire life. The cognitive aspect *Property* reflects such hierarchy. They also discuss intensional defining, which is stating the properties an entity should have to belong to a category. This is also explained by Stamper [43]: an intensional definition starts with the genus (aka headword) and has descriptors for differentia, *i.e.*, *Properties*, which clarify how the defined term differs from the genus, *e.g.*, “a car is a vehicle with at least four wheels”.

5.9 Analogy

An Analogy expresses that a Concept is like another Concept, *i.e.*, the base Concept. The analogous Concept has *Properties* that are like the *Properties* of the base Concept. For example, “a tree with branches is like an animal with limbs”. “Tree” is the analogous Concept, and “animal” is the base Concept. “Branch” is the analogous *Property* and “limb” is the base *Property*.

As mentioned in Sect. 5.4, David Hume stated that similarity is one of the principles of connections between ideas. Analogy reflects the specification of similarity. Stating that something belongs to a Category is also an expression of similarity.

Hofstadter and Sander [38] dedicated a whole book to analogies as the core of mental processes. They conclude that category making and analogy making are really the same mental process. You cannot do one without doing the other. However, they demonstrate that each term has a different usage in making statements. An Analogy is expressed as a relation between two Concepts. A Category is expressed as an abstraction of a set of Concepts.

5.10 Symbol

A Symbol is something that can be perceived and uttered by the human mind and refers to a Concept. The separation of symbol and concept is frequently discussed. Pinker [24] reflects on the computational theory of mind [44] of Alan Turing.⁵ The brain stores and processes information. Information itself is emotionless, it is just symbols. Peoples’ minds give it meaning. Hofstadter and Sander [38] discuss that learning new concepts happens through words and experiences. The former is seeing a word and then learning its meaning. However, people learn concepts mostly through experiences; some of those concepts get their own (compound) word. Levitin [22] addresses the lexical hypothesis, *i.e.*, the most important things humans need to talk about eventually become encoded in language. Hofstadter [45]

even claims that people have a mental symbol for each concept that they know. This Symbol is triggered whenever someone thinks about the concept.

Symbols can be represented in different forms, such as a word, an icon, a sign, a photo, a sound, etc. The most common Symbol form in models is a word or phrase combined with a Symbol that refers to a modeling concept, *e.g.*, a class, use case, or activity in UML. Notice that Symbol is not a subclass of Concept, because it does not have meaning by itself. It only refers to a Concept. A Symbol that identifies an Entity is an identifier.

5.11 Statements and Statement Context

Symbols are used to make Statements about Concepts in a Statement context, which is depicted in Fig. 3. A Statement is a Concept that contains Symbols that refer to the Concepts the Statement is about. The Statements are written in a language that is part of the Statement’s context. The grammar of the language, in combination with the Symbols, makes a Statement interpretable.

To support analysis and design, MuDForM distinguishes Descriptive statements from Prescriptive statements. Descriptive statements are about Concepts as they are (perceived). They are used for analysis, *i.e.*, understanding something. Prescriptive statements state what should be true for the referred Concepts. They are used for design, *i.e.*, they state something that should be realized. The distinction is not driven by a primary cognitive notion, but by the need to have a specification language for developing systems, which involves understanding a system’s context through analysis, and describing the system through requirements and design. For example, “a car has a color” is a Descriptive statement, and “my car must be blue” is a Prescriptive statement.

Statement resembles the concept speech act, as discussed, for example, by John Searle [36]. He explains different types of speech acts, such as declarations, assertives, and directives. A model states what kind of Concepts exist; a model can be seen as a set of declarations. Assertives make a claim about what is going on, which is covered by Descriptive statements. Directives indicate what should be done, which is covered by Prescriptive statements.

Stamper [43] discusses real defining vs. lexical defining. Lexical defining is the attempt to describe how a word is used and what it means in a particular setting, which corresponds to making a Descriptive statement. Real defining is the attempt to declare the nature of something by defining it, which corresponds to making a Prescriptive statement.

⁵ https://en.wikipedia.org/wiki/Alan_Turing.

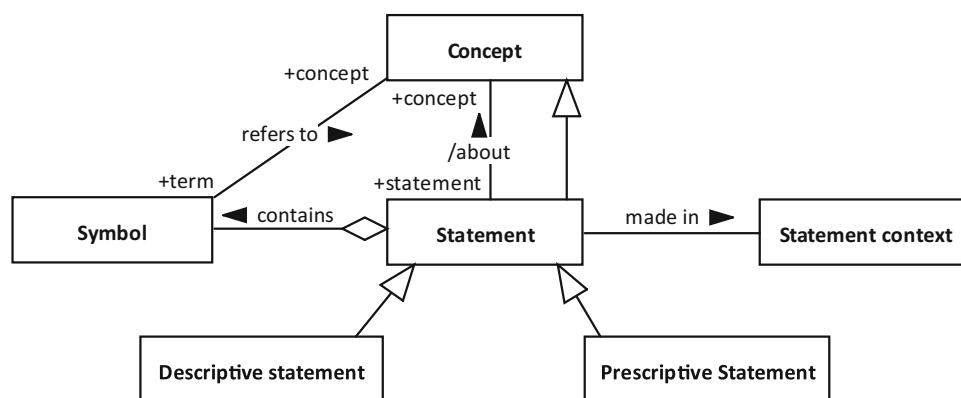


Fig. 3 Statements (UML class diagram)

The term *Statement* is not used in the literature on linguistics and cognition. Authors use other terms to refer to the act of uttering or codifying knowledge and decisions, *e.g.*, *speech act*, *concept*, *symbol*, *idea*, *sentence*, *word*, and *phrase*. We have chosen *Statement* because it expresses that someone is uttering something. This is not limited to words in natural language. For example, source code in a programming language or diagrams in a modeling language can be seen as *Statements*.

The next section explains how the cognitive aspects in this section are used to define modeling concepts.

6 Method definition

This section explains the MuDForM definition by using the elements of Fig. 1 and the cognitive aspects of Figs. 2 and 3. Section 6.1 explains the main structure of a MuDForM model. Section 6.2 explains the construction patterns used in the definition of the metamodel. Section 6.3 explains how the metamodel is defined. Section 6.4 explains the main method steps for transforming natural language into unambiguous models.

6.1 Main Model Structure: Domain, Feature, Context

MuDForM uses the concept of specification space to organize and manage models, similar to packages in UML. MuDForM distinguishes three types of spaces: domains, features, and contexts. These *modeling concepts* are based on the *cognitive aspects* of Fig. 3 in Sect. 5.11.

We found two different notions of **domain** in the literature. The notion that we use is that of a statement space, analogous to a domain in the mathematical sense. The term domain refers to an area of knowledge or activity, and a domain model describes what can happen (behavior) and what can exist (state) in a domain; in other words, what can be controlled and managed in a domain. A domain model is

not intended to express what should happen, does happen, is likely to happen, or has always happened in the domain. In general, approaches for DSLs, *e.g.*, [17, 46, 47], comply with this notion of domain. A domain model is the foundation for a shared lexicon between stakeholders, can serve directly as a structured vocabulary for making other specifications, and can form the underlying model of a DSL. For example, classes in a Banking *domain*⁶, such as *Customer* and *Bank account*, can be used in other specifications to restrict or control their instances, or can serve as the foundation of a DSL's abstract syntax. Instances of the modeling concept *Domain* form a *Statement context* for *Descriptive Statements*, which clarifies how the modeling concept *Domain* is grounded in the cognitive aspects.

The other notion of domain in the literature is a collection of related systems. According to this notion, a domain model defines a set of system features that are common in the domain of interest. This notion is used, for example, by FODA [4]. ODM [2, 48] also adheres to this notion and speaks of descriptive and prescriptive aspects of a domain. MuDForM puts these aspects in different specification spaces because they evolve differently. *Domains* (the descriptive aspect) are more stable than *features* (the prescriptive aspect).

Lee *et al.* [49] identified different meanings of the term **feature** in the literature. MuDForM follows the definition of the ODM approach [48]: A feature is defined as a distinguishable characteristic of a “concept”, *e.g.*, artifact, area of knowledge, system, that is relevant to some stakeholders. Furthermore, MuDForM covers the internal specification of *features*, *i.e.*, not only the signature of a *feature*, but also its working. Consequently, a MuDForM feature model describes what must happen (behavior) and what must exist (state) when the feature is active. Instances of the modeling concept *Feature* form a *Statement context* for *Prescriptive statements*, which clarifies how the

⁶ From here on, *method ingredients*, *i.e.*, *modeling concepts*, *method steps*, *guidelines*, and *viewpoints*, are written in *italics sans serif font*.

modeling concept *Feature* is grounded in the cognitive aspects..

Domain model elements can be used as the types of elements in a feature model. For example, a feature model prescribes how traffic lights at a crossing must be positioned above driving lanes and how they must behave, which can be used in the requirements specification for a traffic light control system. The domain model describes that you can turn on and off the lamps in a traffic light, that you can hang lights above the driving lanes, and that a crossing can have several driving lanes. As such, the domain model serves as a specification space for the feature model.

A model practically always requires elements whose meaning originates outside the domain or feature of interest, hence, do not belong to the corresponding domain or feature model. MuDForM provides the notion of **context** to capture assumptions and knowledge about elements that are needed to specify domains and features, but that exist outside those domains and features. By explicitly specifying the contexts on which a feature or domain depends, domain and feature models have no implicit semantics. Ambiguities can only be caused by unclarity in context models.

Context models can be seen as descriptive, because they define which concepts can be used from outside a domain or feature. They can also be seen as prescriptive, because they declare what should exist outside a domain or feature, *i.e.*, what the environment should provide to the modeled domains and features.

Domain models, feature models, context models, and their relationships make up the main structure of a MuDForM model. For example, for a toll-road payment system, the **domain model describes** that vehicles can arrive and pass a tollgate, someone can open and close the gate, and someone can pay a fee for a passing. The **feature model prescribes** that when a vehicle arrives at a tollgate, someone pays for its passing, the tollgate opens, the vehicle passes the tollgate, and the tollgate closes. This ensures that there will be no open payments, at the expense of blocking tollgates when someone cannot pay. The **context model declares** external concepts, such as payment service, fee, and license plate, to the extent required to specify the elements in the feature and domain model.

The domain model in the example is relatively stable because it describes properties that always hold. Feature models change more often, because desired behavior changes over time. For example, a change could be that someone must pay after the vehicle passes the tollgate, in favor of traffic flow, but with the risk of unpaid fees. Then, the feature model is adapted, but the domain model remains the same.

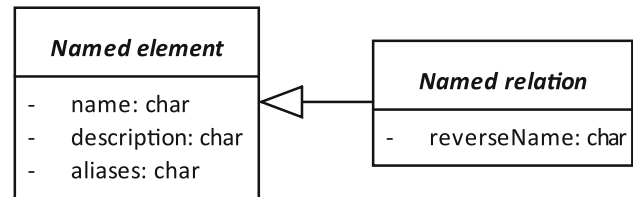


Fig. 4 Named elements (UML class diagram)

6.2 Construction patterns

- This section introduces the two *construction patterns* that are *applied to* define the *modeling concepts* in the *metamodel* (see Fig. 1).

6.2.1 Pattern for naming model elements

The first pattern is for naming model elements. To make models usable for people, model elements must have a humanly understandable symbol, *e.g.*, a name, to identify them. We mention this simple pattern, because it is used to cover the *cognitive aspect* Symbol (Sect. 5.10) and its relation identifies Entity (Sect. 5.3).

Figure 4 shows the two base classes for naming model elements. The *Named element* class has attributes for the *name*, a *description*, and a list of possible *aliases*, *i.e.*, synonyms. The *Named relation* class also has an attribute *reverseName* to define how a relation—all relations are directed—is called in the reverse direction. *Named element* and *Named relation* are used as base classes, as demonstrated in Figs. 5 and 6, respectively.

Pinker [23] and Flanigan [50] state that homonyms are plentiful in natural languages and real synonyms are rare. Although a dictionary might list two words as synonyms, there is mostly a slight difference in their meaning or in the context that they are used.

In a specification, synonyms are more frequent, because not all differences between the meanings of two terms might matter in the specification scope. Homonyms and real synonyms should be avoided, because they lead to unclear communication. However, synonyms must be allowed when different people in different contexts use different terms for the same identity, because they have a different perspective. For example, employee and employment refer to different properties, but may have the same identity, because one does not exist without the other. Vice versa, different people might use the same word, but refer to a different identity. For example, the marketing department uses “book” to refer to the book title, *i.e.*, the specification of the book, and the delivery department uses “book” to refer to the instance that is shipped. The method step Grammatical Analysis (see

Sect. 6.4) has explicit guidelines to eliminate homonyms and synonyms.

Although the semantics of a model are completely determined by its metamodel, a model should be understood by people whose knowledge is captured in the model and by people who must understand the model. This is reflected in the modeling guidelines *A term must be recognized by the expert* and *Involved experts choose names*. There are also guidelines for coming up with suitable names for different types of model elements [28].

6.2.2 Pattern for structures of model elements

The second pattern, or rather, set of patterns, is for defining dependency relations between modeling concepts. There is a dependency when the meaning of one model element is based on the meaning of another model element. It reflects the *cognitive aspect* Property (Sect. 5.8). The relation *involves* (Sect. 5.4) can also be seen as a dependency, *i.e.*, the definition of an Event is dependent on the definition of the Concepts involved in the Event.

The Coordinated structure, Static structure, and Flow structure patterns are defined as template classes with parameters that are bound when applied in the definition of a modeling concept, which is demonstrated in Figs. 5 and 6 of Sect. 6.3. Appendix A gives a detailed explanation of the structure patterns.

Pinker [24] observes that all human cultures ever documented have words for logical connectives, *e.g.*, “not”, “or”, and, “and”, which is reflected in the coordinated structure pattern. The Structure patterns provide a mechanism for specifying possible combinations in a structure of model instances via logical operators. They ensure that models are uniformly structured.

6.3 Metamodel

This section explains how the MuDForM *metamodel* is defined, following the structure of Fig. 1. It demonstrates how *its modeling concepts are based on cognitive aspects and how construction patterns are applied*.

If a MuDForM model is specified correctly, *i.e.*, it complies with the metamodel, then it always has a defined meaning. This is a significant difference from natural language, where grammatically correct sentences can be meaningless. This is nicely demonstrated through Noam Chomsky’s famous sentence ‘Colorless, green ideas sleep furiously’ [51], which is grammatically well-formed, but semantically nonsensical.

For the sake of space, we present here only two metamodel fragments: the modeling concepts *Specification element* and *Specification space*, depicted in Fig. 5, and the modeling concepts *Domain activity* and *Domain class*, depicted in Fig. 6.

These diagrams are excerpts of the complete metamodel [28] and demonstrate the patterns of Sect. 6.2. Appendix B defines all major modeling concepts.

Figure 5 shows that here are three types of *Specification spaces*: *Domain*, *Feature*, and *Context*. A *Specification space* has a *Specification declarations* structure that contains *Specification elements*, which are *Named elements*, and has a *Specification space dependencies* structure that contains *Specification space dependency* instances. A *MuDForM model* is the root of a complete model and has a *Specification declarations* structure that contains only *Specification spaces*.

Figure 6 presents the modeling concepts for the relation between domain activities and domain classes. Each *Domain activity* has a *Role structure*, which is a *Static structure*, where the *static elements* are bound to⁷ *Activity roles*, the *references* are bound to *Involvements*, and the *referred items* are bound to *Domain classes*. *Activity roles* are *Named relations*. An *Involvement* is the connection of a *Domain class* to an *Activity role*, which means that an instance of the *Domain class* may participate in an instance of the *Activity role* of an instance of its *Domain activity*.

A *Domain class* has an *Object lifecycle*. An *Object lifecycle* is a *Flow structure*, in which the *referred items* are *Involvements*. It specifies the possible life of instances of a *Domain class*. A *Flow step* in the *Object lifecycle* specifies when the *Domain class* instance may participate in an instance of the *Activity role* of the corresponding *Involvement*.

Table 1 explains the definition of some modeling concepts of Figs. 5 and 6, following the structure defined in Fig. 1: for each *metamodel class*, *i.e.*, *modeling concept*, it lists the *parent classes*, the *applied construction patterns*, and the *cognitive aspects* on which it is *based*, and it provides an example from the toll-road payment system of Sect. 6.1.

6.4 Main method steps and guidance

The model structure and the various modeling concepts come to life through the *method steps* and *guidelines* (see Fig. 1), which makes MuDForM an actual method and not just a language. As described by the problem statement (Sect. 1.1) and the MuDForM vision (Sect. 2), the objective addressed by this paper is to have a method that goes from analyzing knowledge and decisions expressed in natural language to engineering a model in a language that is based on cognitive concepts. This objective is supported by the main steps of MuDForM:

1. **Scoping** defines the purpose, boundaries, and input text. The goal of scoping is to have relevant input for the model-

⁷ Indicated by the <<bind>> stereotype on the UML dependency relation.

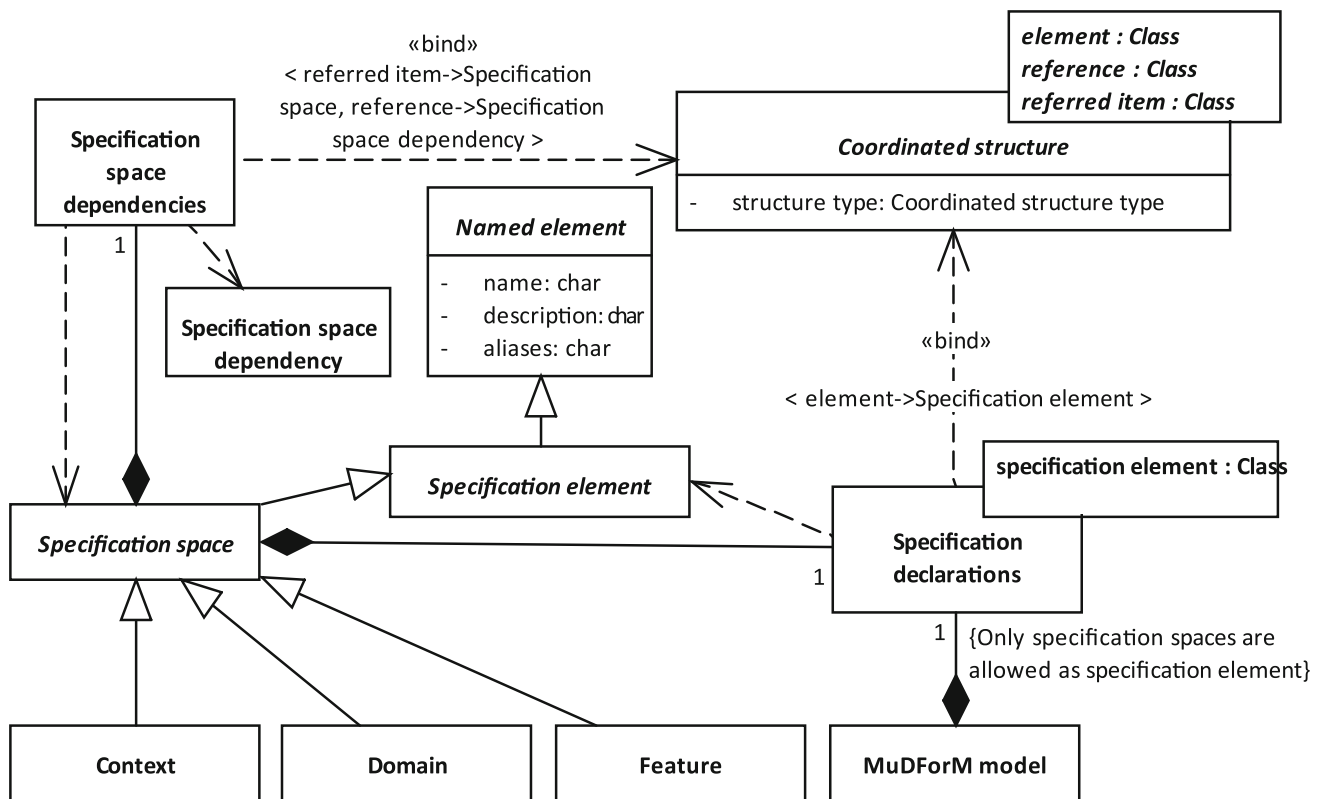


Fig. 5 The metamodel regarding specification spaces (UML class diagram)

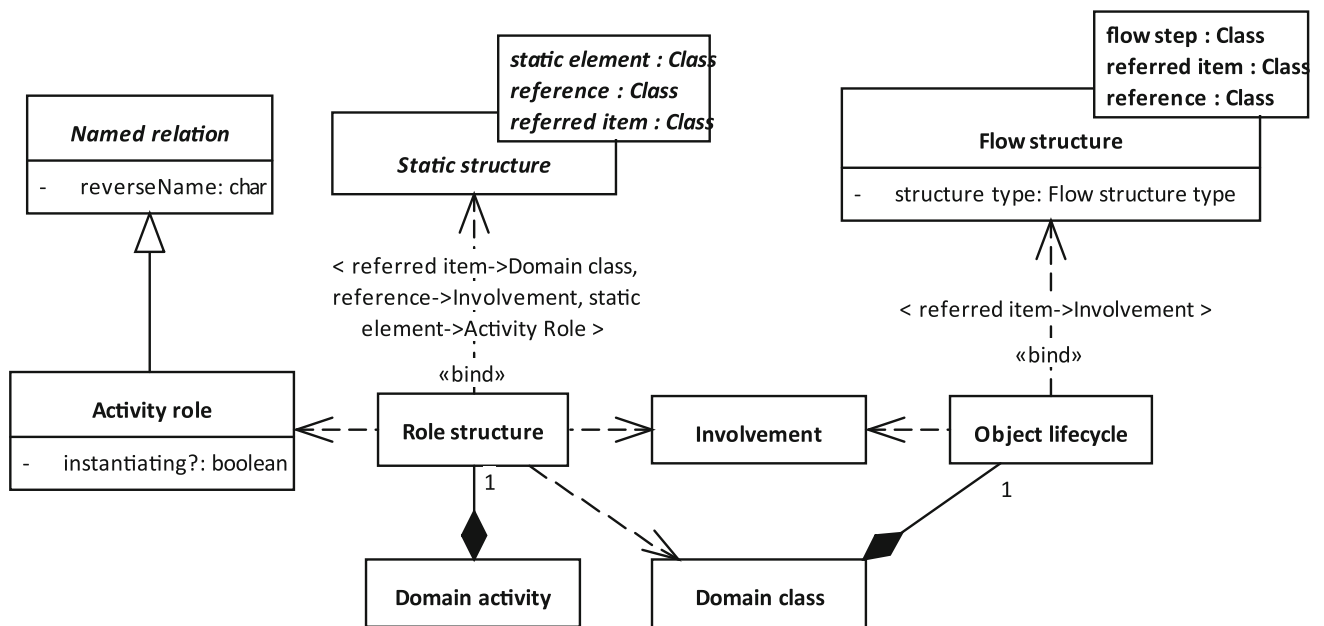


Fig. 6 Metamodel fragment regarding domain classes and domain activitierters (UML class diagram)

Table 1 Examples of modeling concept definitions

Metamodel class	Parent classes	Applied construction patterns (Pattern (elements) in this font)	Based on cognitive aspects (Aspect in this font)	Examples from toll-road payment system
<i>Specification space</i>	<i>Specification element</i>	1. has <i>Coordinated structure</i> <i>Specifications declarations</i> where: <i>element</i> $\rightarrow$ <i>Specification element</i>. 2. has <i>Coordinated structure</i> <i>Specification space dependencies</i> where: <i>reference</i> $\rightarrow$ <i>Specification space</i> <i>dependency</i>, <i>referred</i> <i>item</i> $\rightarrow$ <i>Specification space</i>	The <i>Specifications declarations</i> structure realizes the name space aspect of Statement context. The <i>Specification space dependencies</i> express which other <i>Specification spaces</i> form the Statement context of <i>Specification elements</i> in the <i>Specification space</i>	Domain Toll-road has <i>Specification declarations</i> with <i>Specification elements</i> Arrive, Pass, Open, Close, Pay, Tollgate, Vehicle, and Passing. <i>Domain Toll-road</i> has <i>Specification space dependencies</i> with a <i>Specification space dependency</i> referring to Context Payment Service and a <i>Specification space dependency</i> referring to Context Vehicle registrations
<i>Domain class</i>	<i>Domain element, Class</i>	1. has <i>Flow structure</i> <i>Object lifecycle</i> where: <i>referred item</i> $\rightarrow$ <i>Involvement</i> 2. has <i>Static structure</i> <i>Object life dependencies</i> where: <i>reference</i> $\rightarrow$ <i>Object life dependency</i>, <i>referred item</i> $\rightarrow$ <i>Class</i>	Is used to model Objects. The <i>Object lifecycle</i> expresses the order relation. The <i>Object life dependencies</i> express dependency Properties between <i>Domain classes</i>	<i>Domain class</i> Tollgate has <i>Object lifecycle</i> with <i>Flow steps</i> referring to Open tollgate and Close tollgate. <i>Domain class</i> Passing has <i>Object life dependencies</i> with an <i>Object life dependency</i> referring to <i>Domain class</i> Tollgate and an <i>Object life dependency</i> referring to <i>Domain class</i> Vehicle
<i>Domain activity</i>	<i>Domain element, Activity</i>	1. has <i>Static structure</i> <i>Role structure</i> where: <i>element</i> $\rightarrow$ <i>Activity role</i>, <i>reference</i> $\rightarrow$ <i>Involvement</i>, <i>referred item</i> $\rightarrow$ <i>Domain class</i>. <i>Activity role</i> is a <i>Named relation</i>. 2. has <i>Flow structure</i> <i>Activity flow</i> where: <i>flow step</i> $\rightarrow$ <i>Activity operation</i>, <i>reference</i> $\rightarrow$ <i>Operation invocation</i>, <i>referred item</i> $\rightarrow$ <i>Operation</i>	Is used to model Actions. The <i>Role structure</i> expresses the involves relation. The <i>Activity flow</i> expresses the order of the <i>Operations</i> that are a property of the <i>Domain activity</i>	<i>Domain activity</i> Arrive has a <i>Role structure</i> with an <i>Involvement</i> referring to <i>Domain class</i> Vehicle, and an <i>Involvement</i> referring to <i>Domain class</i> Tollgate. <i>Domain activity</i> Arrive has an <i>Activity flow</i> with a <i>Flow step</i> referring to <i>Operation</i> Check license plate, which is a <i>Specification element</i> in the Context Vehicle registrations

- ing process, in order to (i) prevent unnecessary modeling work, (ii) detect other relevant input, and (iii) keep the model and the modeling process manageable. All relevant domains in a development process can be considered, such as security, user interaction, reliability, and IT networks.
2. **Grammatical analysis** elicits and structures knowledge from the input text. The goal of this step is to maximize the knowledge elicitation from the source and to make the resulting model traceable to the input. A previous paper [10] explains this step in more detail.
 3. **Text-to-model transformation** creates an initial model from the structured text. The goal is to move from natural language analysis to modeling. This step is also detailed in [10].
 4. **Model engineering** ensures that all model aspects are covered and that all inconsistencies are resolved. The goal is to acquire an unambiguous specification that can be understood by stakeholders, either directly or by translating it back to natural language. The model engineering steps that pertain to feature modeling are explained in detail in [11].

The main steps are broken down into sub-steps, amounting to 33 steps in total. Each step has a textual description, which is written in terms of modeling concepts. Each step has guidelines, and possibly preconditions and postconditions. Guidelines help to achieve the postconditions, or help the modeler to perform the step in a work process with involved (domain) experts. Currently, MuDForM has about 125 guidelines, which, together with all steps, are described in the complete method definition [28].

7 Related work

The work related to this paper can be found in three fields: literature on domain-oriented specification methods that also offer an explicit metamodel and method steps, literature on generic method engineering techniques, and literature on method engineering from a cognition-based perspective.

7.1 Domain-oriented specification methods

Deckers and Lago [14] conducted an SLR of domain-oriented specification techniques from the perspective of the MuDForM method objectives. They observed that none of the methods found has a complete method definition, *i.e.*, comprises an underlying model, notation, steps, and guidance. Some approaches provide a metamodel and a notation, and others provide high-level steps, and sometimes guidance.

Most of the studies included in the SLR use UML [35] or MOF [52] as their underlying model. Some have their own metamodel, such as KerMeta [53] or MEMO ML [54,

55]. Most metamodels limit themselves to structural concepts such as classes, attributes, and relations between classes. Only ORM [56], Normalized Systems [57], and the KISS method [3, 29, 30] also provide behavioral concepts.

The KISS method, from which MuDForM is derived, does not provide a metamodel, fine-grained method steps, and detailed modeling guidelines. We have explicitly defined those ingredients for MuDForM. To manage large specifications, we have extended the KISS method with the concept of specification spaces, which supports handling multiple domains, features, contexts, and their relations.

7.2 Generic method engineering techniques

Brinkkemper [58] introduces method engineering in 1996 as a research framework for information systems development methods and tools. He provides definitions of the terms method and technique, to which this paper adheres. Brinkkemper also introduces situational method engineering, *i.e.*, when a method is created from fragments of other methods, mainly for a specific project. Although MuDForM is not created for a specific situation or project, it is clearly based on existing method fragments.

In another study, Brinkkemper *et al.* [59] distinguish five granularity levels for method fragments: *method*, *stage*, *model*, *diagram*, and *concept*. The MuDForM definition can be seen at all these levels. The *stages* (similar to method steps) *Scoping*, *Grammatical analysis*, *Transformation of text to model*, and *Model engineering*, form the backbone of the MuDForM method flow. It is clear that *models*, *diagrams* (similar to viewpoints), and *concepts* are present in the MuDForM definition. Consequently, the MuDForM definition is usable at all five granularity levels.

In relation to this paper, MuDForM offers several techniques for situational method engineering. First, it is possible to replace the method step of transforming natural language into a model with another way to collect usable input from documents and experts, *e.g.*, by analyzing existing Gherkin scenarios [60], textual requirements, or user interfaces. The prerequisite is that such a stage produces elements that can be classified as a MuDForM modeling concept, such as *Domain class* or *Domain activity*. Second, it is possible to use model structures and structural concepts of other methods by (partially) replacing the structure patterns defined in Sect. 6.2.2, *e.g.*, replacing the structure of an object lifecycle with a state transition diagram, or replacing the generalization structure of classes with the corresponding part in the UML definition [35].

7.3 Cognition-related research

This section relates this paper to the various fields of cognition research in software engineering. Section 7.3.1 reflects

on the relation between the model of Sect. 5 and other models of cognition in the literature. Section 7.3.2 reflects on the field of cognitive informatics. Section 7.3.3 relates this paper to other types of cognition-related research in software engineering.

7.3.1 Models of cognition

Evermann [26] studied a cognitive foundation for knowledge representation and created a model of cognitive concepts that resembles the cognitive aspects model in Fig. 2. The model is based on linguistic studies, focusing on the structural analysis of grammars and languages. Evermann discusses the use of ontologies to capture knowledge and concludes that ontology techniques do not reflect human cognition. Ontologies are primarily declarations of concepts and relationships within a domain, and can be used as vocabularies.

Evermann follows the assumption of Jackendoff [61] that ‘the grammatical structure of natural language offers an important source of evidence for the theory of cognition’. Consequently, Evermann’s model is grounded entirely in linguistics. We now discuss the main differences with the cognitive aspects model of Sect. 5.

First, Evermann’s model uses entity as the top-level concept. An entity can either be a spatial entity or a temporal entity, which makes sense from a language perspective because a word cannot be both a noun and a verb in one sentence. For example, the phrase “The help I gave you yesterday” uses a noun, *i.e.*, a spatial entity, and the phrase “I helped you yesterday” uses a verb, *i.e.*, a temporal entity, to refer to the same instance of helping. Evermann’s model sees this as two different entities, maybe because identity is not explicitly recognized as a cognitive concept. In our model, an Entity, *i.e.*, a Concept with an identity, can also be an Event.

Second, Evermann’s model has concepts for location and motion, *i.e.*, go, place, and path, which the cognitive aspects model does not have. Location and motion can be covered via Property and Event, *i.e.*, a Property location can be a characteristic of another Concept and an Event motion can involved another Concept. However, a more direct support makes sense, which is elaborated in Sect. 8.2.

Third, Evermann’s model has a different structure, *i.e.*, different relations between the concepts. It is not clear to us what the rationale for the structure of Evermann’s model is. We organized our model in such a way that the cognitive aspects can be used independently in the definitions of modeling concepts.

Evidently, the main difference with Evermann’s work is that it does not address how the model can be used to create the metamodel of a specification language.

Merril *et al.* [62–64] suggest that almost all content in cognitive subject matter (knowledge) can be represented as

four types of knowledge objects, *i.e.*, entities, actions, processes, and properties. They present a model containing these objects, which is called a process knowledge structure. They might be right about their suggestion, but they do not underpin why the objects and the structure are optimal from a cognitive perspective. For example, we believe that it lacks some important concepts such as state, causation, and agency. It is unclear how they can be captured through the mentioned knowledge objects and structure.

The problem statement in Sect. 1.1 refers to the work of Sia [18]. He proposes to use cognitive psychology as a reference discipline for method engineering. But he does not concretely state how it can be used to define method ingredients. The problem statement also refers to the work of Wand [19] on ontology-based metamodeling. He distinguishes ontological elements and metamodel elements, but does not state what the proper ontological elements are or how they can be found. The modeling concepts of MuDForM are based on explicit cognitive aspects, which demonstrate the use of knowledge from cognitive psychology, as Sia suggests. We think that the cognitive aspects can be seen as elements of an ontology of human cognition, which fits the idea of Wand.

7.3.2 Cognitive informatics

The research field of Cognitive Informatics, introduced by Wang [65] in 2002, studies the internal information processing mechanisms of the brain and their engineering applications through an interdisciplinary approach. He expected that the investigation into cognitive informatics will, among other things, result in new software development approaches. Since then, Wang *et al.* [66, 67] published several papers that discuss the research field.

Wang and Patel [66] explore the cognitive foundations of software engineering. They perceive that the nature of software and software engineering is closer to cognitive psychology than to technology and engineering. Wang and Patel also state that the human brain is the limiting factor in software development productivity: ‘The fact that before any program is composed, an internal abstract model must be created inside the brain, reveals the most fundamental constraint of software engineering, *i.e.*, software is generated and represented in the brain before it can be transferred into the computer. Because the growth rate of the human neural system is naturally constrained, as described by the 24-hour law [68, 69], it is very hard to dramatically improve the productivity of software development.’ They say, to improve software development productivity, software behavior and architecture must be described in denotational mathematics [70], and tools to automatically generate code must be used. Wang *et al.* [67] define denotational mathematics as: ‘a category of expressive mathematical structures that deals

with high-level mathematical entities beyond numbers and sets, such as abstract objects, complex relations, behavioral information, concepts, knowledge, processes, and systems’.

The above perception of software engineering and how to improve its productivity is consistent with the MuDForM vision explained in Sect. 2. The cognitive aspects model and its use in the definition of MuDForM’s metamodel match the idea of formal linguistics from Chomsky [51, 71] and Wang [72]. Although the metamodel is intended to be unambiguous and formal, it is currently not formally specified in the mathematical sense and cannot be classified as denotational mathematics. A formal specification of the semantics of the metamodel is future work.

7.3.3 Cognition-related research in software engineering

There are several tertiary studies on cognition in software development.

Fagerholm *et al.* [73] provide a taxonomy of cognitive concepts and a survey of the literature since the beginning of the Software Engineering discipline. The taxonomy comprises the top-level concepts of perception, attention, memory, cognitive load, reasoning, cognitive biases, knowledge, social cognition, cognitive control, and errors, and procedures to assess them both qualitatively and quantitatively. Only the concept knowledge overlaps with this paper, but Fagerholm *et al.* do not reveal insights about how knowledge is specified.

Anu *et al.* [74] conducted a systematic review of the literature on software engineering and psychology to identify and classify human errors that occur during requirements engineering. They developed the Human Error Taxonomy by adding detailed error classes to Reason’s [75] well-known human error taxonomy of Slips, Lapses, and Mistakes. Quite some error types are related to wrong or missing terminology or to an unclear scope of the requirements domain at hand. They conclude that software engineering can benefit from closer collaboration with cognitive psychology researchers.

Mohanani *et al.* [76] conducted a systematic mapping study on cognitive biases in software engineering. The results reveal a scarcity of research on mitigation techniques. The ones found in the studied literature are all about the organization of work processes.

Kaur *et al.* [77] conducted a systematic literature study related to several cognitive concepts, such as psychology, understanding of program comprehension, and emotion of developers.

Gonçalves *et al.* [78] conducted a systematic mapping study on the cognitive load of software developers. They studied many publications on measuring cognitive load. They do not mention ways to reduce it.

Dawson [79] and Hadar [80] investigated cognitive issues related to object-oriented specifications. They both conclude

that object-oriented models are not always easy to make, because the object-oriented paradigm does not match cognition and intuition. MuDForM has strict steps and rules for deciding what must be a class, attribute, or association, and when to use generalization, which simplifies the modeling activity and reduces the dependency on the modeler’s expertise.

Each of the studies above assumes a traditional development process with requirements, design, programming, and testing phases. They did not consider a process in which people are guided by a method for capturing decisions and knowledge in their own domain language to create specifications. The mentioned studies focused either on the individual developer or team of developers, or on the organization of the development process. They did not study the use of strictly defined (specification) methods to guide the development process.

8 Discussion

This section reflects on the method definition structure, the cognitive aspects, and the method definition.

8.1 A more cognition-based method definition

The method definition structure explained in Sect. 4 is an extension of the method definition framework described by Kronlöf [27]. We have added the concepts *Method objective*, *Cognitive aspect*, and *Construction pattern*. For now, only the definition of *Modeling concepts* is explicitly cognition-based. We believe that also the definition of some *Method steps* and *Guidelines* can be based on cognitive concepts. For this purpose, the structure of Fig. 1 must probably be extended with other types of cognitive concepts besides cognitive aspect. These types of concepts could be derived from the literature mentioned in Sect. 7.3.3, such as perception, attention, memory, cognitive load, reasoning, cognitive biases, social cognition, cognitive control, and human errors. Some of these concepts might also be used to define more detailed method objectives, *e.g.*, reducing the cognitive load of modelers or domain experts during specific method steps.

Kevin Flanigan [50] explains five principles of vocabulary learning; one of them is morphology. The use of different word forms, *i.e.*, morphology, helps to understand what kind of word a specific word is. English uses word endings *e.g.*, -ness, -ability, -ism, and conjugations of verbs. Morphology helps writers choose words and readers understand them. The MuDForM definition prescribes viewpoints, but does not prescribe a syntax. The relations between modeling concepts could be exploited syntactically to increase the usability of models.

We also think that MuDForM's fine-grained method steps, viewpoints, and guidelines can reduce the cognitive load of modelers and experts, because it helps to focus on different aspects at different moments.

At this point, it is unclear how effective the mentioned suggestions will be. Their investigation is future work, as well as researching how the foundation of MuDForM could be strengthened through other types of cognitive concepts.

8.2 Completeness of cognitive aspects

Section 6.3 explains how modeling concepts are based on construction patterns and cognitive aspects and presents a few examples. Appendix B shows that almost each cognitive aspect in Sect. 5 is used in the definition of at least one modeling concept. In addition, all modeling concepts are related to at least one cognitive aspect, which underpins that the metamodel is cognition-based. We applied MuDForM in many cases, *e.g.*, [8–11], but we did not quantitatively measure the closeness to cognition. To make closeness to cognition a stronger claim, research is needed *e.g.*, to determine how well the modeling concepts resonate with people who have to make and understand models. A starting point could be the cognitive concepts addressed by the related literature in Sect. 7.3.3 and determine whether MuDForM is an improvement.

The cognitive aspect *Analogy* (see Sect. 5.9) is currently not supported by a dedicated modeling concept. An analogy can be achieved by identifying a model fragment that is the base of the analogy and then replacing its model elements with the analogous elements. We think that having specific modeling concepts and method steps for making analogies would be an improvement to MuDForM worth investigating.

Pinker [24, 37, 40] observes that location in space and movement are fundamental metaphors in many languages. As mentioned in Sect. 7.3.2, it makes sense to have cognitive aspects for location and motion. For example, by extending the diagram of Fig. 2 in the following way. The cognitive aspect *Location* is a subclass of *Concept*, with an association *Concept is at Location*. *Motion* is a subclass of *Event* with an association *Motion of Concept*, and optional associations *Motion from Location* and *Motion to Location*. This is a starting point, but maybe *Location* and *Motion* must have more distinct properties. The extension of the cognitive aspects model with location and motion, the definition of derived modeling concepts in the metamodel, and their evaluation is future work. The evaluation of the modeling concepts could be initiated by using some of the previous case studies [8–11]. Model fragments from those cases could be replaced or extended with model fragments that use modeling concepts related to *Analogy*, *Motion*, and *Location*.

8.3 Experience with the method definition

The structure patterns (see Sect. 6.2) are used throughout the entire metamodel, which brings several benefits. They provide expressive relations between metaclasses, dealing with aspects like multiplicities, nesting, ordering, and (conditional) selection between the elements in an N-ary relation. They provide uniformity in relations between metaclasses, which simplifies their definition, representation, and implementation in software. The disadvantage is that sometimes not all possible instances of a used pattern are allowed, which requires an extra constraint to express the restriction. See, for example, the constraint *Only specification spaces are allowed as specification element* in the diagram of Fig. 5. We could have modeled the declarations of a MuDForM model in a separate structure that only contains specification spaces, and thus, no extra constraint is required. But then it is unclear that its essence matches the declarations in other specification spaces, *i.e.*, being a container of specification elements.

In addition to modeling concepts, we also found that some MuDForM guidelines can be considered cognition-based (see the background report [13]). We give two examples.

Pinker [24] observes that people have to get something out of using categories, *i.e.*, inference. If you know a category of a thing, then you should be able to infer that that thing has the properties of the category. The guideline *Ensure consistency between classifiers and known instances* corresponds to the observation about inference.

Corballis [39] writes about the notions of episodic memory and semantic memory. Episodic memory is the memory of events or episodes in our lives. Semantic memory is memory for statements about the world. Involved experts can be asked to share general knowledge about the target domain or system, which can be used directly as model input. However, this can be hard. The guideline *Ask for examples when generic knowledge is hard to phrase* appeals to the episodic memory of the expert involved.

We also observed that MuDForM supports the mitigation of some errors and biases mentioned by the works of Anu *et al.* [74] and Mohanani *et al.* [76] in Sect. 7.3.3. For example, if you capture requirements in feature models that are specified in terms of domain models, then those requirements are unambiguous. This reduces the chance of errors in understanding the meaning of requirements. Furthermore, a domain model can be treated as the problem space for a requirement specification. This could reduce the availability bias⁸ of people providing and collecting requirements input, because the domain model can be used to check if all domain elements occur in at least one requirement.

⁸ https://en.wikipedia.org/wiki/Availability_heuristic, *e.g.*, explained in [81].

9 Conclusion and future work

This paper describes the definition of a specification method called MuDForM. We were interested in addressing the following research question: *How to define a specification method that is explicitly cognition-based?*

By providing such a cognition-based method definition, the results from our study fill an important gap in the state of the art, which, to the best of our knowledge, does not yet provide specification methods that are explicitly cognition-based. To create the method definition, we introduced a method definition structure in which modeling concepts can be explicitly based on cognitive specification aspects. Through qualitative data analysis, we identified such aspects and explained how they form the foundation for MuDForM's modeling concepts.

The current method definition lays the foundation for future work on the following topics:

- Relate the cognitive concepts found in related work, e.g., attention, perception, memory, cognitive load, to the method steps of MuDForM, and analyze if they can be used to improve MuDForM.
- Develop and evaluate guidelines that explicitly mitigate cognitive biases and human errors.
- Evaluate the actual proximity to cognition of MuDForM by applying it to case studies.
- Investigate what modeling concepts are useful for specifying analogies, and location and motion, and evaluate them in case studies.
- Develop and evaluate viewpoints and syntax rules that reduce the cognitive load of modelers and domain experts.
- Make a formal specification of the semantics of the metamodel, such that it fits the notion of denotational semantics as described by Wang [69].
- The current set of cognitive aspects is derived from a large range of literature, which was found through qualitative data analysis. It would be better to use explicit criteria to identify cognitive aspects and to decide whether they should be used in a method definition.

All of these topics contribute to the goal of making software development a cognition-aligned process. We believe MuDForM is a good step forward, which we experience in practice, but there is room for strengthening the cognitive foundation.

Appendix A All structure patterns

This section is an extension of Sect. 6.2.2 and describes each of the patterns for creating relations between model elements.

The root pattern is **Structure** (see Fig. 7). A *Structure* consists of *Elements*. An *Element* can be, but not necessarily, a *Substructure* that consists of other *Elements*, or a *Reference* to a *Referred item*. A *Substructure* is itself a *Structure*. This enables the creation of a tree structure, where the nodes are *Elements*, and the leaves, i.e., *References*, can refer to other model elements, i.e., the *Referred items*.

The structure pattern is used as the foundation for more specific structures, such as the *Roles* of a *Domain activity*, and the *Object lifecycle* of a *Domain class*. To achieve this, the class *Structure* is declared as a template class. The parameters of the class correspond to the variable part of the pattern. The parameters *element*, *reference*, and *referred item* can be bound to specific classes when applying the *Structure* class.

A **Coordinated structure** (see Fig. 8) is meant to constraint the instances of *Elements* of instances of *Structures*. The *structure type* defines which instances of *Elements* an instance of the *Coordinated structure* must or can contain. For example, the modeling concept *Domain class* has a *Coordinated structure* for the *Attributes*. If an *Domain class* Person has attributes name and registration code, then several cases can be expressed via the *structure type* attribute of *Coordinated structure*. If a Person must have a value for:

- name **and** code, then the *structure type* equals “All of” (AND).
- name **and/or** code, then the *structure type* equals “Some of” (OR).
- *either* a name **or** a code, then the *structure type* equals “One of” (XOR).

A **Static structure** is a *Coordinated structure*, where the parameter *element* is bound to the class *Static element* (see Fig. 8). The attribute *how many?* specifies the possible number of instances of a *Static element*. For example, the modeling concept Class has a *Static structure* for the attributes of a Class. Then, if you define a Class CarType that has:

- Exactly one value for an *Attribute* driverSeatType, then *how many?* equals “One”. (This is the default value.)
- Possibly several values for an *Attribute* doorType, then *how many?* equals “Zero or more”.
- Possibly a value for an *Attribute* roofType, then *how many?* equals “At most one”.
- An *Attribute* for specifying one or more colors, *how many?* equals “At least one”.

A **Flow structure** is used to describe a temporal order of instances of *Referred items* (see Fig. 8). A *Flow structure* is a *Structure* where the *element* parameter of the template class *Structure* is bound to the class *Flow step*. The order of the *Flow steps* is specified via the value of the attribute *structure*

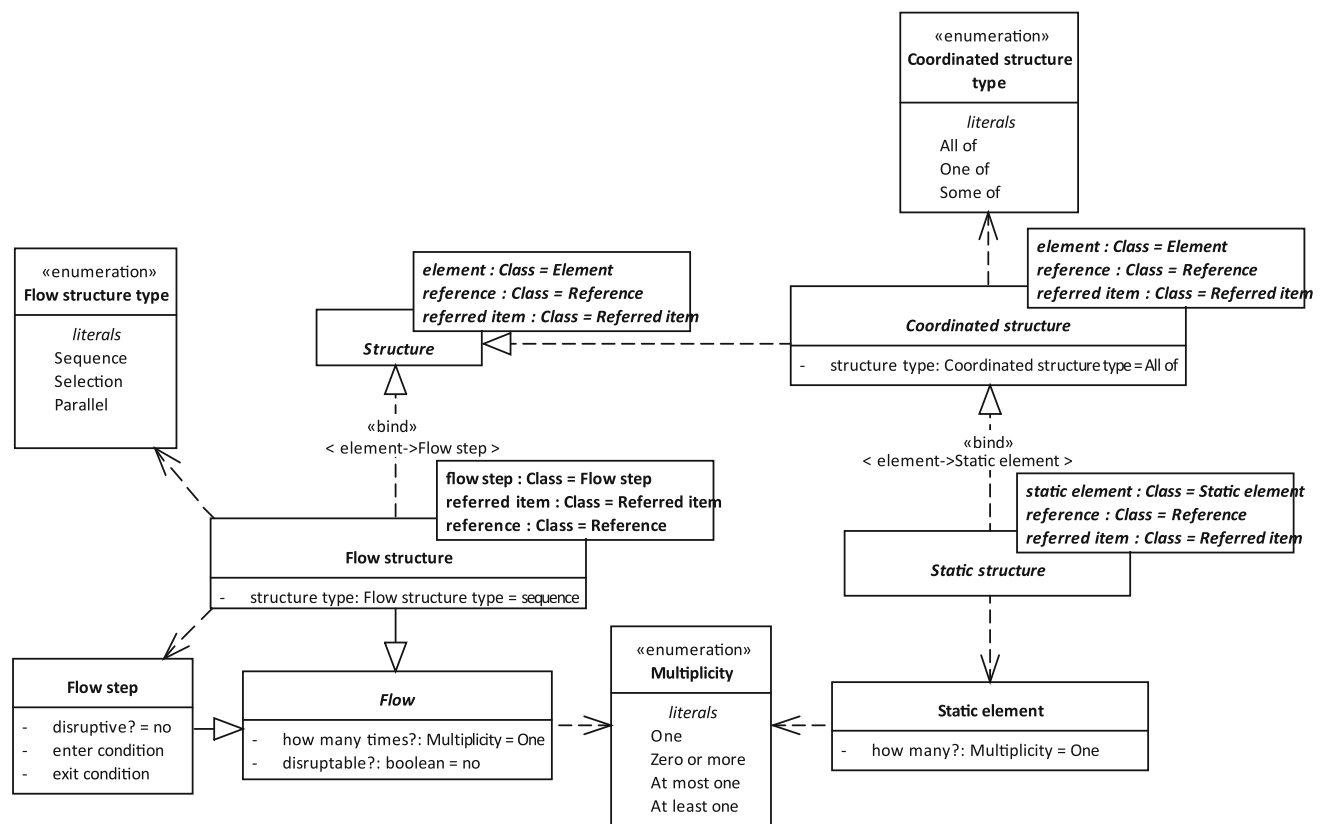


Table 2 Modeling concepts construction based on cognitive aspects

Metamodel class	Parent classes	Applied construction patterns (Pattern (elements) terms in this font)	Based on cognitive aspects (Aspect in this font)
Generic concepts			
<i>Specification element</i>		<i>Specification element</i> is a <i>Named element</i>	The name of a model element, together with its type, <i>i.e.</i> , the modeling concept, realize the aspect Symbol. The relation <i>identifies</i> is realized by ensuring that the combination of model element name and type is unique in its scope, <i>i.e.</i> , the Statement context
<i>Specification space</i>	<i>Specification element</i>	1. has <i>Specifications declarations</i>, which is a <i>Coordinated structure</i> where: <i>element</i> → <i>Specification element</i>. 2. has <i>Specification space dependencies</i>, which is a <i>Coordinated structure</i> where: <i>reference</i> → <i>Specification space dependency</i>, <i>referred item</i> → <i>Specification space</i>	The <i>Specifications declarations</i> structure realizes the name space aspect of Statement context. The <i>Specification space dependencies</i> express which other <i>Specification spaces</i> form the Statement context of <i>Specification elements</i> in the <i>Specification space</i>
<i>MuDForM model</i>		has <i>Specifications declarations</i> , which is a <i>Coordinated structure</i> where: <i>element</i> → <i>Specification element</i> . Only <i>Specification spaces</i> are allowed as <i>Specification element</i> of a <i>MuDForM model</i>	Via the <i>Specification declarations</i> structure, a <i>MuDForM model</i> forms a Statement context for the contained <i>Specification spaces</i>
<i>Classifier</i>		has a <i>Generalization structure</i> , which is a <i>Coordinated structure</i> where: <i>reference</i> → <i>Generalization</i> , <i>referred item</i> → <i>Classifier</i>	All modeling concepts are based on the aspect Category, because the only practical application of a model is that it can be instantiated multiple times. The model elements that are a <i>Specification element</i> in a <i>Declarations</i> structure, all have the abstract class <i>Classifier</i> as a parent class. The <i>Generalization structure</i> is based on the aspect Property, as it expresses that one Category, <i>i.e.</i> , the <i>Generalization</i> , is a characteristic of another Category
<i>Attribute container</i>		has an <i>Attribute structure</i> , which is a <i>Static structure</i> where: <i>reference</i> → <i>Attribute</i> , <i>referred item</i> → <i>Class</i> . <i>Attribute</i> is a <i>Named element</i>	Expresses the aspect Property in case the characteristic is a <i>Class</i>
<i>Class</i>	<i>Classifier</i>		Based on Entity, <i>i.e.</i> , it represents a Category of identified Concepts

Table 2 continued

Metamodel class	Parent classes	Applied construction patterns (<i>Pattern (elements) terms in this font</i>)	Based on cognitive aspects (<i>Aspect in this font</i>)
Generic concepts			
<i>Activity</i>	<i>Classifier</i>	has a <i>Behavior composition</i> , which is a <i>Static structure</i> where: <i>reference</i> → <i>Activity reference</i> , <i>referred item</i> → <i>Activity</i>	Based on Event. The <i>Attribute structure</i> expresses the involves relation in case the involved Concept is a <i>Class</i> . The <i>Behavior composition</i> expresses a <i>Property</i> in case the characteristic is an <i>Activity</i>
Context concepts			
<i>Context</i>	<i>Specification space</i>	has <i>Context declarations</i> , which is a <i>Specification declarations</i> , where <i>specification element</i> → <i>Context element</i>	Represents a Statement context for Statements needed to make Statements in other Statements contexts
<i>Class</i>	<i>Context element</i> , <i>Attribute container</i>		To represent an Entity from a <i>Context</i>
<i>Value type</i>	<i>Class</i>	has attributes <i>type</i> and possible <i>ValueSpec</i> of type <i>Value type</i> , which can be defined via a used <i>Formalism</i>	Is used to express a single value Entity that exists outside the <i>Domain</i> or <i>Feature</i> of interest. The possible <i>ValueSpec</i> specifies the Entities that are identified by such values
<i>Operation</i>	<i>Context element</i> , <i>Activity</i> , <i>Attribute container</i>	has an attribute <i>formula</i> of type <i>Formula</i> defined via a used <i>Formalism</i>	Represents an Event from a <i>Context</i>
<i>Class relation</i>	<i>Context element</i> , <i>Classifier</i>	<i>Class relation structure</i> , which is a <i>Static structure</i> , where <i>element</i> → <i>Class relation role</i> , <i>reference</i> → <i>Role connection</i> , <i>referred item</i> → <i>Class</i> . <i>Class relation role</i> is a <i>Named relation</i>	
<i>Actor</i>	<i>Class</i> , <i>Activity</i>		<i>Actor</i> is based on <i>Agent</i> . The <i>Behavior composition</i> express the capabilities of an <i>Actor</i> , and the <i>modality</i> attribute of an <i>Activity reference</i> expresses the performs and perceives relation

Table 2 continued

Metamodel class Generic concepts	Parent classes	Applied construction patterns (Pattern (elements) terms in this font)	Based on cognitive aspects (Aspect in this font)
Domain concepts <i>Domain</i>	<i>Specification space, Attribute container</i>	has <i>Domain declarations</i> , which is a <i>Specification declarations</i> where: <i>specification element</i> → <i>Domain element</i>	Represents a Statement context for Descriptive statements
<i>Domain class</i>	<i>Domain element, Class</i>	1. has an <i>Object lifecycle</i> , which is a <i>Flow structure</i> where: <i>referred item</i> → <i>Involvement</i> 2. has <i>Object life dependencies</i> , which is a <i>Static structure</i> where: <i>reference</i> → <i>Object life dependency, referred item</i> → <i>Class</i>	Is used to model <i>Objects</i> . The <i>Object lifecycle</i> expresses the order relation. The <i>Object life dependencies</i> express hierarchy Properties between <i>Domain classes</i>
<i>Domain activity</i>	<i>Domain element, Activity</i>	1. has a <i>Role structure</i> , which is a <i>Static structure</i> where: <i>element</i> → <i>Activity role</i> , <i>reference</i> → <i>Involvement</i> , <i>referred item</i> → <i>Domain class</i> . <i>Activity role</i> is a <i>Named relation</i> . 2. has an <i>Activity flow</i> , which is a <i>Flow structure</i> , where: <i>flow step</i> → <i>Activity operation</i> , <i>reference</i> → <i>Operation invocation</i> , <i>referred item</i> → <i>Operation</i>	Is used to model <i>Actions</i> . The <i>Role structure</i> expresses the involves relation. The <i>Activity flow</i> expresses the order of the <i>Operations</i> that are a Property of the <i>Domain activity</i>
<i>Feature concepts</i> <i>Feature</i>	<i>Specification space, Function</i>	has <i>Feature declarations</i> , which is a <i>Specification declarations</i> where: <i>specification element</i> → <i>Feature element</i>	Represents a Statement context for Prescriptive statements
<i>Function</i>	<i>Feature element, Activity</i>	1. has a <i>Function lifecycle</i> , which is a <i>Flow structure</i> where: <i>flow step</i> → <i>Function step</i> , <i>referred item</i> → <i>Function event</i> . (<i>Function event</i> is an <i>Activity reference</i>). 2. has a <i>Behavior life dependencies</i> , which is a <i>Static structure</i> where: <i>reference</i> → <i>Behavior life dependency, referred item</i> → <i>Function</i>	<i>Function</i> is a combination of <i>Action</i> and <i>Object</i> . The <i>Function lifecycle</i> expresses the order and causes relations. The <i>Behavior life dependencies</i> expresses which <i>Activities</i> are a Property of the <i>Function</i>

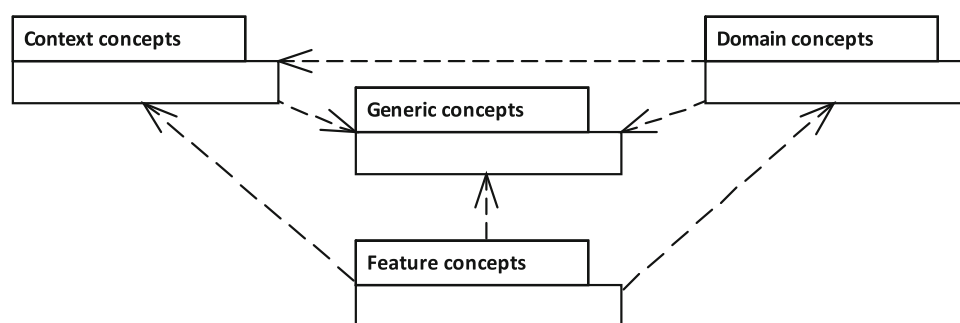


Fig. 9 Structure of the metamodel definition (UML package diagram)

Feature concepts define modeling concepts used in context models, domain models, and feature models, respectively.

Table 2 defines the classes in the metamodel. For each metaclass, the table lists the parent classes, the applied construction patterns of Appendix A, and on which cognitive aspects of Sect. 5 it is based. For the sake of space, the minor modeling concepts and the attributes of the modeling concepts are omitted. The complete metamodel [28] contains all modeling concepts, including a description of each modeling concept.

References

- Shlaer, S., Mellor, S.J.: An object-oriented approach to domain analysis. *SIGSOFT Softw. Eng. Notes* **14**(5), 66–77 (1989). <https://doi.org/10.1145/71633.71639>
- Simos, M.A.: Organization domain modeling (ODM): Formalizing the core domain modeling life cycle. In: *SIGSOFT Software Engineering Notes, Special Issue on the 1995 Symposium on Software Reusability*, pp. 196–205. , New York, USA (1995)
- Kristen, G.: *Object-Orientation: The KISS Method: From Information Architecture to Information System*. Addison-Wesley Educational Publishers Inc, Raleigh, NC, USA (1994)
- Kang, K.C., Cohen, S.G., Hess, J.A., Novak, W.E., Peterson, A.S.: *Feature-oriented domain analysis (FODA) feasibility study*, Software Engineering Institute, Carnegie Mellon University (1990). (Technical report)
- Evans, E.: *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, Boston, MA, USA (2004)
- Frappier, M., Habrias, H.: *Software Specification Methods: an Overview Using a Case Study*. Springer, Heidelberg, Germany (2012)
- Deckers, R.: *MuDForM Repository* (2025). <https://github.com/robertdeckers/MuDForM>
- Moghaddam, F.A., Deckers, R., Procaccianti, G., Grosso, P., Lago, P.: A domain model for self-adaptive software systems. In: *Proceedings of the 11th European Conference on Software Architecture: Companion Proceedings*, pp. 16–22. ACM, New York, NY, USA (2017)
- Saberi, K., A.: *Functional safety: A new architectural perspective: Model-based safety engineering for automated driving systems*, Eindhoven University of Technology (2020). (PhD thesis)
- Deckers, R., Lago, P.: Methodical conversion of text to models: MuDForM definition and case study. In: *Proceedings of the Forum at Practice of Enterprise Modeling 2022 (PoEM-Forum 2022)*, November 23–25. CEUR Workshop Proceedings, vol. 3327, pp. 113–127. CEUR-WS.org, Aachen, Germany (2022). <http://ceur-ws.org/Vol-3327/paper10.pdf>
- Deckers, R., Lago, P.: Specifying features in terms of domain models: MuDForM method definition and case study. *J. Softw.: Evolut. Process* **36**(6), e2623 (2024)
- Deckers, R.: *From smallest software particle to system specification, MuDForM: Multi-domain formalization method*, PhD thesis, VU Amsterdam (2024). <https://doi.org/10.5463/thesis.831>
- Deckers, R.: *Cognition Perspective on MuDForM*. Zenodo (2026). <https://doi.org/10.5281/zenodo.19000193>
- Deckers, R., Lago, P.: Systematic literature review of domain-oriented specification techniques. *J. Syst. Softw.* **192**(C), (2022). <https://doi.org/10.1016/j.jss.2022.111415>
- Clark, T., Sammut, P., Willans, J.S.: *Applied metamodeling: A foundation for language driven development (second edition)*. CoRR abs/1505.00149 (2015)
- Chaudhuri, R., Natarajan, S., Banerjee, S., Choppella, A., V.: *Methodology to develop domain specific modeling languages*. In: *Proceedings of the 17th ACM SIGPLAN International Workshop on Domain-Specific Modeling*, pp. 1–10. ACM, New York, NY, USA (2019)
- Fowler, M.: *Domain-Specific Languages*. Addison-Wesley Professional, Boston, MA, USA (2010)
- Siau, K.: Information modeling and method engineering: A psychological perspective. In: *Successful Software Reengineering*, pp. 193–208. IGI Global, Hershey, PA, USA (2002)
- Wand, Y.: Ontology as a foundation for meta-modelling and method engineering. *Inf. Softw. Technol.* **38**(4), 281–287 (1996)
- Deckers, R., Lago, P., Fokkink, W.: *Towards a human-centric development method* (2025). <https://doi.org/10.5281/zenodo.17184709>
- Sigman, M.: *The Secret Life of the Mind: How Your Brain Thinks, Feels, and Decides*. Little Brown & Co, USA (2017)
- Levitin, D.J.: *The Organized Mind: Thinking Straight in the Age of Information Overload*. Penguin, USA (2014)
- Pinker, S.: *The Language Instinct: How the Mind Creates Language*. William Morrow and Company, New York, NY, USA (1994)
- Pinker, S.: *How the Mind Works*. Penguin UK, London, UK (2003)
- Pinker, S.: *The Stuff of Thought: Language as a Window Into Human Nature*. Penguin, USA (2007)
- Evermann, J.: Towards a cognitive foundation for knowledge representation. *Inf. Syst. J.* **15**(2), 147–178 (2005)
- Kronlöf, K.: *Method Integration: Concepts and Case Studies*. John Wiley and Sons, Hoboken, NJ, USA (1993)
- Deckers, R.: *MuDForM Method Definition* (2024). <https://github.com/robertdeckers/MuDForM>
- Vos, B., Hoppenbrouwers, J., Hoppenbrouwers, S.: NL structures and conceptual modelling: The KISS case. In: *Applications of Natural Language to Information Systems: Proceedings of the Second*

- International Workshop, p. 197. IOS Press, Amsterdam, the Netherlands (1996)
30. Hoppenbrouwers, S., Bleeker, A., Proper, H.: Modeling linguistically complex business domains, Nijmegen Institute for Information and Computing Sciences, University of Nijmegen (2004). (Technical report)
 31. Chomsky, N.: On Language: Chomsky's Classic Works Language and Responsibility and Reflections on Language in One Volume. New Press, New York, NY, USA (1998)
 32. McWhorter, J.H., Leven, J., Blandford, J.: The Story of Human Language. Teaching Company, Chantilly, Virginia, USA (2004)
 33. Searle, J.R.: Philosophy of mind, Lecture 1–28. UC Berkeley Lectures. Accessed: 2021–10–20 (2011)
 34. Jalali, S., Wohlin, C.: Systematic literature studies: Database searches vs. backward snowballing. In: Proceedings of the 2012 ACM-IEEE International Symposium on Empirical Software Engineering and Measurement, pp. 29–38. ACM/IEEE, New York, NY, USA (2012)
 35. OMG: Unified modeling language version 2.5.1. Technical report, OMG (December 2017). <https://www.omg.org/spec/UML/2.5.1/pdf>
 36. Searle, J.R.: Philosophy of language, Lecture 1–28. UC Berkeley Lectures. Accessed: 2020–08–19 (2012)
 37. Pinker, S.: Words and Rules: The Ingredients of Language. Basic Books, New York, NY, USA (1999)
 38. Hofstadter, D.R., Sander, E.: Surfaces and Essences: Analogy as the Fuel and Fire of Thinking. Basic Books, New York, NY, USA (2013)
 39. Corballis, M.C.: A Very Short Tour of the Mind: 21 Short Walks Around the Human Brain. Abrams, New York, NY, USA (2017)
 40. Pinker, S.: The Sense of Style: The Thinking Person's Guide to Writing in the 21st Century. Penguin Books, USA (2015)
 41. Shermer, M.: The Believing Brain: From Ghosts and Gods to Politics and Conspiracies - How We Construct Beliefs and Reinforce Them as Truths. Macmillan, UK (2011)
 42. Sloman, S., Fernbach, P.: The Knowledge Illusion: Why We Never Think Alone. Penguin, USA (2018)
 43. Stamper, K.: Word by Word: The Secret Life of Dictionaries. Vintage, UK (2018)
 44. Turing, A.: On computable numbers, with an application to the entscheidungsproblem. In: Proceedings of the London Mathematical Society, pp. 230–265. (1936)
 45. Hofstadter, D.R.: I Am a Strange Loop. Basic books, New York, NY, USA (2007)
 46. Abouzahra, A., Bézivin, J., Del Fabro, M.D., Jouault, F.: A practical approach to bridging domain specific languages with UML profiles. In: Proceedings of the Best Practices for Model Driven Software Development at OOPSLA, vol. 5, Springer, New York, NY, USA (2005)
 47. Kelly, S., Tolvanen, J.-P.: Domain-Specific Modeling: Enabling Full Code Generation. John Wiley & Sons, Hoboken, NJ, USA (2008)
 48. Simos, M., Creps, R., Klingler, C., Lavine, L.: Software technology for adaptable reliable systems (stars). organization domain modeling (ODM) guidebook, version 1.0. Defense Technical Information Center, (1995) DTIC. (Technical report)
 49. Lee, K., Kang, K.C., Lee, J.: Concepts and guidelines of feature modeling for product line software engineering. In: International Conference on Software Reuse, pp. 62–77. Springer, New York, NY, USA (2002)
 50. Flanagan, K.: Building a Better Vocabulary. The Great Courses, Chantilly, Virginia, USA (2015)
 51. Chomsky, N.: Syntactic Structures. Mouton, The Hague/Paris (1957)
 52. OMG: Meta object facility 2.5.1. Technical report, OMG (October 2016). <https://www.omg.org/spec/MOF/2.5.1/PDF>
 53. France, R., Fleurey, F., Reddy, R., Baudry, B., Ghosh, S.: Providing support for model composition in metamodels. In: 11th IEEE International Enterprise Distributed Object Computing Conference (EDOC 2007), pp. 253–253. IEEE, New York, NY, USA (2007)
 54. Frank, U.: Outline of a method for designing domain-specific modelling languages. Technical report, Universität Duisburg-Essen, Institut für Informatik und Wirtschaftsinformatik (ICB) (2010)
 55. Frank, U.: The memo meta modelling language (mml) and language architecture. Technical report, Universität Duisburg-Essen, Institut für Informatik und Wirtschaftsinformatik (ICB) (2011)
 56. Proper, H.A., Bleeker, A.I., Hoppenbrouwers, S.J.B.A.: Object-role modelling as a domain modelling approach. In: Proceedings of the Workshop on Evaluating Modeling Methods for Systems Analysis and Design (EMMSAD'04), pp. 317–328. CEUR-WS.org, Aachen, Germany (2004)
 57. Mannaerts, H., Verelst, J.: Normalized Systems. Koppa BvBa, Hasselt, Belgium (2009)
 58. Brinkkemper, S.: Method engineering: Engineering of information systems development methods and tools. Inf. Softw. Technol. **38**(4), 275–280 (1996)
 59. Brinkkemper, S., Saeki, M., Harmsen, F.: Assembly techniques for method engineering. In: Advanced Information Systems Engineering: 10th International Conference. CAiSE'98 Pisa, Italy, June 8–12, 1998 Proceedings 10, pp. 381–400. Springer, New York, NY, USA (1998)
 60. Smart, J.: BDD in Action: Behavior-Driven Development for the Whole Software Lifecycle. Simon and Schuster, New York, NY, USA (2014)
 61. Jackendoff, R.: Semantics and Cognition. The MIT Press, Cambridge, MA, USA (1983)
 62. Merrill, M.D.: Knowledge objects and mental models. In: Proceedings International Workshop on Advanced Learning Technologies. IWALT 2000, Advanced Learning Technology: Design and Development issues, pp. 244–246 (2000)
 63. Merrill, M.D., Group, I.R.: Instructional transaction theory: Instructional design based on knowledge objects. Educational Technology, pp. 30–37. (1996)
 64. Merrill, M., Team, I.R.: Instructional transaction theory: Knowledge relationships among processes. Entities, and Activities, p. 19. (1992)
 65. Wang, Y.: On cognitive informatics. In: Proceedings First IEEE International Conference on Cognitive Informatics, pp. 34–42 (2002). IEEE
 66. Wang, Y., Patel, S.: Exploring the cognitive foundations of software engineering. Int. J. Softw. Sci. Comput. Intell. (IJSSCI) **1**(2), 1–19 (2009)
 67. Wang, Y., Kinsner, W., Anderson, J.A., Zhang, D., Yao, Y., Sheu, P., Tsai, J., Pedrycz, W., Latombe, J.-C., Zadeh, L.A., et al.: A doctrine of cognitive informatics (ci). Fund. Inform. **90**(3), 203–228 (2009)
 68. Wang, Y., King, G.: Software Engineering Processes: Principles and Applications. CRC Press, Boca Raton, FL, USA (2000)
 69. Wang, Y.: On the informatics laws and deductive semantics of software. IEEE Trans. Syst. Man, Cybernetics, Part C (Applications and Reviews) **36**(2), 161–171 (2006)
 70. Wang, Y.: On contemporary denotational mathematics for computational intelligence. In: Transactions on Computational Science II, pp. 6–29. Springer, New York, USA (2008)
 71. Chomsky, N.: Aspects of the Theory of Syntax. MIT press, Cambridge, MA, USA (1965)
 72. Wang, Y.: Software engineering foundations: A transdisciplinary and rigorous perspective. In: CRC Book Series in Software Engineering, vol. 2. CRC Press, Boca Raton, FL, USA (2006)
 73. Fagerholm, F., Felderer, M., Fucci, D., Unterkalmsteiner, M., Marculescu, B., Martini, M., Tengberg, L.G.W., Feldt, R., Lehtelä, B., Nagyvárdi, B., et al.: Cognition in software engineering: A taxon-

- omy and survey of a half-century of research. *ACM Comput. Surv. (CSUR)* **54**(11s), 1–36 (2022)
74. Anu, V., Hu, W., Carver, J.C., Walia, G.S., Bradshaw, G.: Development of a human error taxonomy for software requirements: A systematic literature review. *Inf. Softw. Technol.* **103**, 112–124 (2018)
 75. Reason, J.: *Human Error*. Cambridge University Press, Cambridge, MA, USA (1990)
 76. Mohanani, R., Salman, I., Turhan, B., Rodríguez, P., Ralph, P.: Cognitive biases in software engineering: A systematic mapping study. *IEEE Trans. Software Eng.* **46**(12), 1318–1339 (2018)
 77. Kaur, K., Mishra, A., Chopra, P.: An amalgamation of cognitive aspects in software engineering: A content analysis. *Expert Syst. Appl.* **246**, 122968 (2024)
 78. Gonçalves, L., Farias, K., Silva, B., Fessler, J.: Measuring the cognitive load of software developers: A systematic mapping study. In: 2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC), pp. 42–52. IEEE (2019)
 79. Dawson, L.: Cognitive processes in object-oriented requirements engineering practice: analogical reasoning and mental modelling. In: *Information Systems Development: Reflections. Challenges and New Directions*, pp. 115–128. Springer, New York, USA (2013)
 80. Hadar, I.: When intuition and logic clash: The case of the object-oriented paradigm. *Sci. Comput. Program.* **78**(9), 1407–1426 (2013)
 81. Kahneman, D.: *Thinking. Fast and Slow*. Penguin Books Ltd, London, UK (2012)
 82. Fokkink, W.: *Introduction to Process Algebra*. Springer, New York, NY, USA (2013)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Robert Deckers earned his Master of Science and Doctor of Engineering degrees from Eindhoven University of Technology, and PhD degree at Vrije Universiteit Amsterdam. Since 1991, he has focused on specification methods, architecture, code generation, and model interpreters. He has developed and implemented specification languages, modeling techniques, and supporting tools. Robert has extensive experience as a practitioner, teacher, coach, lecturer, and consultant in architecture, do

main modeling, and requirements engineering at all organizational levels. He is the author of the book “DYA|Software, architecture approach for mission critical applications.” In 2013, he founded his own company to concentrate on model-driven development, emphasizing that “software development is about integrating knowledge and decisions, not about realizing technology.” More info available at www.linkedin.com/in/robertdeckers and www.github.com/robertdeckers.



Patricia Lago is a Professor of software engineering in the Software and Sustainability research group and Director of the Digital Sustainability Center at Vrije Universiteit Amsterdam, The Netherlands. Her research focuses primarily on the intersection between software sustainability and software architecture design and decision making, and software quality assessment. She is the recipient of an Honorary Doctorate at NTNU, Norway, for her contribution to the field of software sustainability, and of the 2023 IEEE CS TCSE New Directions Award. She has a PhD in Control and Computer Engineering from Politecnico di Torino and a Master in Computer Science from the University of Pisa, both in Italy. She is a senior member of IEEE, a member of ACM and VERSEN. Contact her at p.lago@vu.nl.